Parte 1 — Identifique conhecimentos críticos

Conhecimento crítico	**Tácito ou explícito?**	**Quem possui?**	**Onde está registrado?**	**Risco de perda**
1. Regras de negócio do sistema legado	Tácito	Desenvolvedor *Senior* / Analista de Negócio	Não está registrado (apenas no código fonte antigo)	**Alto**
2. Arquitetura da API e integrações	Explícito	Arquiteto de Software	Wiki interna (Confluence/Notion) e Swagger	**Médio**
3. Processo de *Deploy* e pipeline de CI/CD	Tácito	Engenheiro DevOps / *Tech Lead*	Scripts locais e na cabeça do DevOps	**Alto**
4. Padrões de código e *Clean Code*	Explícito	Equipe de Desenvolvimento	Documento de boas práticas no Git/GitHub	**Baixo**
5. Resolução de *bugs* recorrentes em produção	Tácito	Suporte N3 / Desenvolvedores	Apenas nos históricos de chamados encerrados	**Médio**

Parte 2 — Priorize os conhecimentos

Conhecimento Prioritário 1: Regras de negócio do sistema legado

- **Por que esse conhecimento é importante?** Garante que novas funcionalidades ou migrações não quebrem regras de cálculo, validações fiscais e fluxos operacionais críticos da empresa.
- **Quem depende dele?** Desenvolvedores, testadores (QA), suporte técnico e a equipa de produtos (*Product Owner*).
- **O que aconteceria se esse conhecimento fosse perdido?** Interrupção nas entregas, introdução de *bugs* críticos em produção, atrasos na modernização do sistema e elevado custo de retrabalho.
- **Existe outra pessoa que também domine esse conhecimento?** Não de forma completa; o conhecimento está pulverizado entre dois colaboradores antigos.
- **Como ele poderia ser preservado ou compartilhado?** Realização de sessões de *Knowledge Transfer* (KT), mapeamento de processos em diagramas de fluxo e documentação formal na wiki da equipa.

Conhecimento Prioritário 2: Processo de *Deploy* e pipeline de CI/CD

- **Por que esse conhecimento é importante?** É responsável por colocar o software atualizado no ar de forma segura, automatizada e sem indisponibilidade para os utilizadores.
- **Quem depende dele?** Toda a equipa de desenvolvimento e a área de operações de TI.
- **O que aconteceria se esse conhecimento fosse perdido?** Impossibilidade de lançar correções urgentes (*hotfixes*) ou novas versões, além de risco de paragem (*downtime*) do sistema por erros manuais durante o *deploy*.
- **Existe outra pessoa que também domine esse conhecimento?** Apenas uma pessoa domina os detalhes de configuração do ambiente de produção.
- **Como ele poderia ser preservado ou compartilhado?** Criação de documentação passo a passo (*Runbooks*), automação de rotinas via *Infrastructure as Code* (IaC) e realização de *pair programming* / treinamentos internos.

Parte 3 — Aplique o modelo SECI

Conhecimento Selecionado: Regras de negócio do sistema legado e arquitetura de sistemas.

Etapa	Sua proposta
Socialização	Realizar sessões semanais de *Pair Programming* (programação em par) e *Shadowing* (um programador júnior/pleno acompanha a rotina do sénior durante a resolução de chamados).
Externalização	O especialista responsável escreve guias técnicos, diagramas de arquitetura (ex: C4 Model) e documentação funcional das regras de negócio na Wiki do projeto.
Combinação	Integrar a documentação criada com os diagramas de entidade-relacionamento (DER), coleções do Postman/Swagger e histórias de utilizador salvas no Jira/Azure DevOps.
Internalização	A equipa estuda a documentação, realiza testes práticos em ambientes de homologação (*sandbox*) e passa a desenvolver novos módulos aplicando as regras aprendidas autonomamente.

Parte 4 — Registre uma lição aprendida

- **Contexto:** Lançamento de uma nova versão do sistema com uma grande atualização na base de dados durante o horário comercial.
- **Problema ou situação:** O script de migração do banco de dados travou devido ao alto volume de acessos simultâneos, gerando indisponibilidade do sistema durante 2 horas.
- **Causa:** Ausência de testes de carga prévios com a base de dados povoada e falta de agendamento do *deploy* para horários de menor tráfego (janela de manutenção).
- **Impacto:** Impossibilidade de os utilizadores realizarem operações, aumento de chamados no suporte e insatisfação dos clientes.

- **O que funcionou:** A equipa ativou rapidamente o plano de *Rollback* (reversão), restabelecendo a versão anterior do sistema.
- **O que não funcionou:** Executar migrações estruturais na base de dados diretamente em horário de pico sem testes de stresse prévios.
- **Lição aprendida:** Toda e qualquer alteração de grande impacto na infraestrutura ou na base de dados deve possuir plano de reversão validado e ser executada fora do horário comercial.
- **Recomendação:** Implementar uma rotina obrigatória de testes de *script* em ambiente de homologação espelhado e estabelecer janelas de *deploy* noturnas ou ao fim de semana para *releases* críticas.
- **Responsável pela atualização:** *Tech Lead* / Engenheiro DevOps.

Parte 5 — Crie o fluxo da lição aprendida

Etapa	Responsável	Ferramenta	Ação principal
Identificar	Qualquer membro da equipa de ADS	Reunião de Retrospectiva da *Sprint* / Post-mortem	Sinalizar falhas, gargalos ou boas práticas observadas no ciclo de desenvolvimento.
Registrar	Scrum Master / Analista de Sistemas	Formulário de Lições Aprendidas (Confluence/Notion)	Preencher o modelo com contexto, causa, impacto e recomendação.
Validar	*Tech Lead* / Arquiteto de Software	Reunião de Revisão Técnica	Avaliar se a lição é pertinente e se a recomendação proposta é

Etapa	Responsável	Ferramenta	Ação principal
			tecnicamente viável.
Classificar	Analista de Qualidade (QA)	Tags no Confluence (ex: #bancodedados, #deploy, #seguranca)	Categorizar o conhecimento para facilitar a busca futura por outros membros da equipa.
Publicar	*Product Owner* / Scrum Master	Base de Conhecimento do Projeto	Disponibilizar a lição no repositório central de documentação do repositório.
Compartilhar	*Tech Lead*	Slack / Teams / Reunião Geral de TI	Divulgar o aprendizado com a equipa através de um breve resumo nas reuniões diárias (*Dailies*).
Reutilizar	Desenvolvedores / Analistas	Checklist de *Definition of Done* (DoD) do Jira	Consultar a lição durante o planeamento de novas tarefas para evitar a repetição do erro.

Etapa	Responsável	Ferramenta	Ação principal
Atualizar	Engenheiro DevOps / *Tech Lead*	Wiki Técnica	Revisar e atualizar a recomendação à medida que novas tecnologias ou processos forem adotados.